Neovim desde cero

Este texto es el cuaderno del curso. La idea no es memorizar 200 atajos el primer día. Es hablar el idioma de Vim y, cuando eso ya no duela, usar esta configuración como IDE.

Espacio es el leader. Si lees `<leader>ff`, pulsa Espacio y luego `ff`. Si pulsas solo Espacio y esperas, which-key te enseña el menú. Espacio dos veces quita el resaltado de una búsqueda; no abre el buscador de archivos.

Cuando dudes de un atajo de esta config: `:Telescope keymaps`.

`:help` es el manual de verdad. Este cuaderno no lo sustituye.

Objetivos y ruta de aprendizaje

Cada tema sigue el mismo camino, de menos a más:

1. **Presentación** (en línea o course/slides): la idea y los comandos, con ejemplos comprobados en Neovim real.

2. **Práctica** (página de práctica): ejercicios paso a paso en el navegador, con la misma numeración que los capítulos 1 a 7.

3. **Configuración real** (capítulos 8 a 13): instalar esta config y usarla como IDE, con IA y en IntelliJ.

Ningún capítulo usa un comando que no se haya enseñado antes.

Al terminar podrás editar/deshacer con operadores, manejar buffers/ventanas, repetir tareas con registros/macros y comprobar LSP/plugins en un proyecto. Esta secuencia didáctica parte de los manuales oficiales `usr_02` y `usr_04`. Es una decisión del curso, no una valoración sobre otros autores.

Capítulos 1-7: usa `nvim --clean archivo.txt` para aprender sin mapas del repo. Después repite con `nvim archivo.txt`. `jj`, búsquedas centradas y atajos con leader se identifican como añadidos. Así no se confunden `H`/`L` nativos con Shift-h/Shift-l de bufferline.

`boot-ok` comprueba solo el arranque. Pruebas y límites: docs/VALIDATION.md. IdeaVim requiere una sesión del IDE.

0. Manos en el teclado

Practicar sin mirar ayuda a localizar teclas. `h j k l` están en la fila central de QWERTY.

Practica 15 minutos al día, sin este editor:

• keybr.com: te va soltando letras.

• monkeytype.com: velocidad.

• typingclub.com: lecciones (hay español).

Cuando ya no busques la `J` con los ojos, abre openvim.com (tutorial en el navegador) o, mejor, el tutor que trae Vim:

```
nvim --clean +Tutor
```

(En Vim clásico el comando es `vimtutor`. En Neovim: `nvim --clean +Tutor`.)

Haz el tutor entero una vez. Aburre. Funciona.

Entrenador de escritorio (Windows / Linux / macOS, sin red): carpeta `trainer/`. `flutter test` y `flutter run`. No es Neovim; solo el subconjunto documentado en Acerca de. Builds de GitHub **sin firma**.

1. Modos y sobrevivir

Vim no es un bloc de notas con atajos. Es un editor **modal**: la misma tecla hace cosas distintas según el modo.

Estás en…	Entras con	Sales con	Sirve para
Normal	al abrir, o `Esc`	(es el modo base)	moverte y dar órdenes
Insertar	`i a I A o O`	`Esc` o `jj`	escribir
Visual	`v V Ctrl-v`	`Esc`	seleccionar
Línea de comandos	`: / ?`	`Enter` o `Esc`	guardar, buscar, ayuda

Si te pierdes: `Esc Esc`. El pitido (o nada) significa “ya estás en Normal”.

En esta config, `jj` también sale de insertar. `Esc` sigue existiendo. Aprende los dos.

Ejercicio. `nvim prueba.txt` -> `i` -> escribe tu nombre -> `Esc` -> `i` otra vez -> `jj`. Mira la esquina: la statusline dice el modo (esta config oculta `-- INSERT --` nativo; lualine lo muestra).

Guardar, salir y pedir ayuda

Comando	Qué hace
`:w`	guardar
`:q`	salir si no hay cambios
`:wq`	guardar y salir (siempre escribe)
`ZZ`	guardar y salir; solo escribe si hay cambios (igual que `:x`)
`:q!`	salir tirando los cambios
`:e!`	recargar el archivo del disco
`:help`	manual
`:help x`	la tecla `x`
`:help :w`	el comando `:w`
`:help 'number'`	una opción
`Ctrl-]`	seguir un enlace en el help
`Ctrl-t` o `Ctrl-o`	volver

En el help, `Ctrl-d` después de un tema incompleto lista coincidencias.

Practica: lección 1 de la página de práctica

2. Moverse sin flechas

Las flechas funcionan. Te hacen sacar la mano de las letras. El manual de Vim insiste en `h j k l` por eso.

```
    k
  h   l
    j
```

`h` está a la izquierda, `l` a la derecha, `j` apunta abajo.

Tecla	Qué hace
`h j k l`	un carácter / una línea
`w b e`	palabra siguiente, anterior, fin de palabra
`W B E`	igual, pero saltando puntuación
`0 ^ $`	inicio de línea, primer no-espacio, final
`gg G`	primera línea, última
`{ }`	párrafo
`( )`	frase
`fx tx`	en esta línea: hasta la x / antes de la x
`Fx Tx`	lo mismo hacia atrás
`; ,`	repetir `f`/`t`
`%`	el paréntesis, corchete o llave de enfrente
`H M L`	alto, medio, bajo de la **pantalla**
`Ctrl-d Ctrl-u`	media página
`Ctrl-f Ctrl-b`	página

Delante de casi todo puedes poner un **número**: `5j` baja cinco líneas, `3w` tres palabras, `10G` va a la línea 10.

Ejercicio. Abre cualquier texto largo. Ve al final con `G`, al principio con `gg`, a la línea 20 con `20G`. Recorre un párrafo con `}` y vuelve con `{`. Busca una coma en la línea con `f,`.

Practica: lección 2 de la página de práctica

3. La gramática: operador + movimiento

Esta es la base de la edición. Un **operador** espera un **movimiento** (o un objeto de texto).

Operadores que vas a usar todos los días:

Operador	Significado
`d`	borrar (delete)
`c`	borrar y entrar a insertar (change)
`y`	copiar (yank)
`> <`	indentar / desindentar
`=`	reindentar según indentexpr/cindent; no equivale a formatear con LSP
`gU gu g~`	mayúsculas, minúsculas, invertir

Movimiento = dónde. Entonces:

• `dw` borrar hasta el inicio de la siguiente palabra

• `d$` borrar hasta el final de la línea

• `dG` borrar hasta el final del archivo

• `c3w` cambiar tres palabras

• `yip` copiar el párrafo interior (esto ya es objeto de texto; el siguiente capítulo)

Atajos que parecen comandos sueltos pero son esta gramática con un movimiento especial _ (la línea actual): `dd`, `cc`, `yy`. El tutor lo enseña como “borrar línea”; por debajo es `d` sobre la línea.

Otros que conviene tener en los dedos:

Tecla	Qué hace
`x`	borrar un carácter (`dl`)
`X`	borrar el de atrás (`dh`)
`s`	cambiar un carácter. **Ojo:** en esta config, `s` es Flash (saltar). Para “change char” usa `cl`
`r` + letra	reemplazar un carácter sin entrar a insertar
`J`	juntar esta línea con la de abajo
`p P`	pegar después / antes
`.`	repetir el último cambio
`u`	deshacer
`Ctrl-r`	rehacer

Insertar con intención:

Tecla	Dónde inserta
`i`	antes del cursor
`a`	después del cursor
`I`	al primer no-espacio de la línea
`A`	al final de la línea
`o`	línea nueva debajo, ya en insertar
`O`	línea nueva encima

Ejercicio. Escribe tres líneas. `dd` la del medio. `p` la pega debajo. `u` la devuelve. `3j` no hace falta: estás en un archivo corto. Cambia una palabra con `cw` (escribe otra, `Esc`). Pulsa `.` en otra palabra: Vim repite el cambio.

Practica: lección 3 de la página de práctica

4. Objetos de texto (antes de cualquier plugin)

Un movimiento va en una dirección. Un **objeto** es una cosa alrededor del cursor: una palabra, una frase, lo que hay entre comillas.

Empiezan por `i` (inner, por dentro) o `a` (around, con el envoltorio).

Objeto	Ejemplo	Efecto
`iw aw`	`diw`	borra la palabra, con o sin espacio pegado
`is as`	`cis`	cambia la frase
`ip ap`	`yip`	copia el párrafo
`i" a"`	`ci"`	cambia lo que hay entre comillas
`i' a'`	`da'`	borra comillas simples y contenido
`i( a(`	`ci(`	cambia dentro de paréntesis (también `ib`)
`i{ a{`	`da{`	borra el bloque `{ ... }`
`i[ a[`	`vi[`	selecciona dentro de corchetes
`it at`	`cit`	dentro de un tag HTML/XML
`i< a<`	`ci<`	dentro de <>

La receta que más se usa al programar: `ciw` (cambiar esta palabra), `ci"` (cambiar el string), `ci{` (cambiar el bloque).

Visual: `v` carácter, `V` líneas, `Ctrl-v` bloque (columnas). Con la selección hecha, `d c y >` actúan sobre ella. `o` salta al otro extremo de la selección.

En esta config, `>` y `<` en visual **mantienen** la selección (para indentar varias veces).

Ejercicio. Escribe `foo("hola mundo")`. Cursor en `hola`. `ci"` -> escribe `adiós` -> `Esc`. Cursor en `foo`. `ciw` -> `bar`. `va(` y mira qué se selecciona.

Practica: lección 4 de la página de práctica

5. Buscar y sustituir

`/palabra` Enter busca hacia adelante. `?palabra` hacia atrás. `n` siguiente, `N` anterior. En esta config, `n`/`N` además **centran** la pantalla.

`*` busca la palabra bajo el cursor hacia adelante, `#` hacia atrás.

`:nohl` quita el resaltado. Aquí: `<leader><leader>`.

Sustituir (línea de comandos):

```
:s/viejo/nuevo/        « solo la primera vez en esta línea
:s/viejo/nuevo/g       « todas en esta línea
:%s/viejo/nuevo/g      « todo el archivo
:%s/viejo/nuevo/gc     « todo el archivo, preguntando
```

`%` es el rango “archivo entero”. `:'<,'>s/` (sale solo si venías de visual) actúa sobre las **líneas** seleccionadas, no solo sus columnas. Para limitar a la zona visual, consulta `:help /\%V`.

Ejercicio. Pon tres veces la palabra `gato`. `:%s/gato/perro/g` y comprueba.

Practica: lección 5 de la página de práctica

6. Registros y macros

Yank (`y`) no es “el portapapeles de Windows” nada más. Hay cajones:

• `"` sin nombre: último yank o borrado

• `0` último yank

• `1` a `9` últimos borrados

• `a` a `z` los tuyos (`"ayy` copia la línea al registro a)

• `+` portapapeles del sistema (esta config usa `unnamedplus`)

• `_` agujero negro (`"_dd` borra sin ensuciar el yank)

`:reg` enseña el contenido.

Macros: `q` + letra para grabar, `q` para parar, `@a` para reproducir, `@@` la última. Ejemplo: en una lista, `qa` -> `I-` Esc `j q` y luego `10@a`.

Ejercicio. Copia una línea con `yy`. Muévete. `p`. Ahora `"ayy` en otra línea y `"ap`.

Practica: lección 6 de la página de práctica

7. Buffers, ventanas, pestañas y terminales

Olvida un momento Telescope. En Vim:

• **Buffer** = un archivo cargado en memoria. Puede no verse.

• **Ventana** = un hueco que mira a un buffer.

• **Pestaña** = un conjunto de ventanas.

`:e archivo` abre (o crea) un buffer. `:ls` lista. `:b siguiente` o `:b 2` cambia. `:bd` cierra el buffer.

Ventanas nativas:

Tecla	Qué hace
`Ctrl-w s`	partir horizontal
`Ctrl-w v`	partir vertical
`Ctrl-w h/j/k/l`	ir a esa ventana
`Ctrl-w c`	cerrar ventana
`Ctrl-w o`	dejar solo esta

Esta config añade (mismo significado, con leader): `<leader>sv` vertical, `sh` horizontal, `sc` cerrar, `so` solo esta, `wh wj wk wl` moverse.

Buffers con pestañas de bufferline: `Shift-l` / `Shift-h` siguiente/anterior, `<leader>bn bp bd`.

Saltos. Cada vez que haces un salto grande (`G`, `/`, `gd` más adelante), Vim guarda el sitio. `Ctrl-o` vuelve atrás, `Ctrl-i` adelante. `:jumps` lista. `<leader>b` en esta config es “atrás” de historial (como el Back del IDE); espera 300 ms porque `bn`/`bp` empiezan igual.

Marcas. `ma` guarda la posición en la marca `a`. `'a` vuelve al inicio de esa línea, `` `a `` a la columna exacta. Marcas `A` a `Z` son globales (otro archivo).

Ejercicio. Abre dos archivos con `:e`. Parte la ventana `Ctrl-w v`. Salta con `G`, vuelve con `Ctrl-o`.

Terminales (esta config). `<leader>tv` abre una terminal a la derecha y `<leader>th` abajo, como paneles de tmux; con un número delante abres otra (`2<leader>th`). Dentro, `jj` vuelve a modo Normal y `Ctrl-h/j/k/l` cambia de ventana; `i` vuelve a escribir en la terminal. `<leader>ta` oculta o muestra todas sin cerrar sus procesos y `<leader>tl` elige una.

Practica: lección 7 de la página de práctica

8. Instalar esta config

Neovim **0.12 o más**; referencia probada: **0.12.6**. Requisitos y compiladores: README.md. Respalda una configuración existente antes de enlazar. No clones el repo dentro de `~/.config/nvim`.

Windows

```
winget install --id Neovim.Neovim --exact
winget install --id Git.Git --exact
winget install --id DEVCOM.JetBrainsMonoNerdFont --exact
# tree-sitter CLI >= 0.26.1: binarios oficiales o Cargo; consulta README.md
$nvimRepo = Join-Path $HOME "Neovim-Vim_Configuration"
git clone https://github.com/chochy2001/Neovim-Vim_Configuration.git $nvimRepo
$nvimLink = @{
    ItemType = "Junction"
    Path = "$env:LOCALAPPDATA\nvim"
    Target = "$nvimRepo\.config\nvim"
}
New-Item @nvimLink
Copy-Item "$nvimRepo\.ideavimrc" "$HOME\.ideavimrc"
```

En el terminal pon fuente **JetBrainsMono NFM** y ciérralo. Sin esa fuente los iconos salen como recuadros.

macOS / Linux

```
# neovim, git, node, compilador C, ripgrep, fd: con brew o el paquete de la distro
# tree-sitter CLI >= 0.26.1: binarios oficiales o Cargo; consulta README.md
git clone https://github.com/chochy2001/Neovim-Vim_Configuration.git ~/Neovim-Vim_Configuration
mkdir -p ~/.config
ln -s ~/Neovim-Vim_Configuration/.config/nvim ~/.config/nvim
ln -s ~/Neovim-Vim_Configuration/.ideavimrc ~/.ideavimrc
```

Primera vez: `nvim` y espera a que lazy.nvim instale. `:Lazy` para mirar. Mason instala servidores de lenguaje en segundo plano.

Comprobar arranque:

```
nvim --headless "+lua print('boot-ok')" +qa
```

Tiene que salir `boot-ok`, sin `Error detected`. Después ejecuta `python scripts/verify.py` desde la raíz; arrancar no prueba los plugins diferidos.

9. Esta config como IDE

Hasta aquí se explica Vim nativo y se señalan las adiciones de esta config. Lo siguiente añade cosas. Si algo no carga, `:Lazy`. Si no hay autocompletado, el LSP de ese lenguaje no está (`:Mason`, `:checkhealth vim.lsp`).

Dashboard: solo si abres `nvim` sin archivo. `f` archivo, `g` grep, `r` recientes, `e` árbol, `a` opencode, `m` Mason, `l` Lazy, `q` salir.

Prefijos (Espacio y una letra):

Prefijo	Familia
`f`	buscar (Telescope)
`g`	git
`b`	buffers (y Back si esperas)
`w`	ventanas
`x`	errores
`m`	harpoon
`t`	terminal
`r`	ejecutar
`d`	depurar
`fl`	Flutter
`a`	terminales de IA (solo Neovim)
`c`	Copilot Chat (solo Neovim)

Neovim 0.12 ya trae de serie, también con `nvim --clean`: `Y` copia hasta el final de la línea (como `y$`), `gcc` comenta la línea, `grn` renombra, `gra` muestra acciones de código, `grr` busca referencias y `[d ]d` saltan entre diagnósticos. Esta config añade sus equivalentes con leader (`<leader>rn`, `<leader>ca`, `gR`, `<leader>xn`), así que funcionan las dos formas.

Flujo típico: `<leader>ff` abre -> `ciw` edita -> `gd` definición (si hay LSP) -> `<leader>fm` formatea (none-ls o LSP) -> `<leader>gs` git.

10. IA con Neovim

Los agentes de IA son programas de terminal (opencode, Codex, Claude, Gemini, Grok, Copilot, muse, Cursor, Antigravity, kimi) que leen tu proyecto y editan archivos. Esta config los abre en una terminal flotante: `<leader>aa` opencode, `<leader>ax` Codex, `<leader>ac` Claude, `<leader>ag` Gemini, `<leader>ak` Grok, `<leader>ap` Copilot, `<leader>am` muse, `<leader>au` Cursor, `<leader>an` Antigravity y `<leader>ai` kimi. Cada uno usa su propia cuenta; la config no guarda claves.

Pedir un cambio sobre código concreto: selecciona en Visual y pulsa `<leader>as`. Eliges el agente y escribes qué quieres ("refactoriza esto", "añade pruebas"). El archivo se guarda y el agente se abre a la derecha, en la raíz del repo, ya trabajando con tu instrucción, el archivo, las líneas y el código seleccionado. Con kimi, o en Windows con CLIs instalados por npm, el texto queda en el portapapeles y en el registro `a`: pégalo, revisa y envía.

El flujo completo de un día de trabajo:

1. `nvim` en la carpeta de tus proyectos; `<leader>fp` abre un proyecto reciente y `<leader>ff` busca un archivo.

2. `<leader>pv` abre el árbol de carpetas: `j`/`k` para moverte, `Enter` abre, `<leader>pv` lo cierra.

3. Lee el código: `gd` va a la definición, `K` muestra la documentación, `gR` lista las referencias y `Ctrl-o` vuelve.

4. Selecciona con `V` y pide el cambio con `<leader>as`.

5. Mira trabajar al agente a la derecha; cuando guarda, el archivo se recarga solo.

6. Revisa: `<leader>gsp` muestra el cambio del bloque, `<leader>gdo` el diff completo y `<leader>gsr` deshace un bloque que no te guste.

Si el agente edita un archivo que tienes abierto, Neovim lo recarga solo, también si el agente corre en otro panel de herdr o tmux. Si tú tenías cambios sin guardar, Neovim avisa y no los pisa: `:e!` toma la versión del disco y `:w` guarda la tuya. Revisa lo que cambió con `<leader>gsp` (vista del cambio) o `<leader>gdo` (diff).

Las reglas para los agentes de este repo están en `AGENTS.md`; Claude y Gemini/Antigravity las leen a través de `CLAUDE.md` y `GEMINI.md`.

11. Plugins de este repo (cuándo, no el catálogo entero)

Tablas largas: WORKFLOW.md. Aquí los atajos abreviados como `fg`, `pv`, `tt` o `cc` llevan **Espacio delante**. Atajos sin leader: `gcc`, `gd`, `K`, `s`, `af`; varios dependen de LSP o plugins.

which-key: Espacio y espera. Es el índice.

telescope: `<leader>ff` archivo, `fg` texto (necesitas ripgrep), `fo` recientes, `fk` atajos. `,,` también busca archivos.

neo-tree / oil: `pv` árbol. `-` o `oe` editar la carpeta como si fuera un buffer (renombrar archivos escribiendo).

harpoon: archivos de cabecera del día: `<leader>ma` añade, `<leader>1` a `<leader>9` saltan.

Comment / surround / flash: `gcc` comenta la línea. Surround: `ysiw"` pone comillas a la palabra, `ds"` las quita, `cs"'` cambia `"` por `'`. Flash: `s` salta a un carácter. **Visual** `S` **es surround, no Flash.**

treesitter: resaltado basado en la estructura del código. En visual/operador: `af`/`if` función, `ac`/`ic` clase (cuando el parser está). Eso es además de `iw`/`i"`.

LSP + cmp + none-ls: `gd` definición, `K` documentación, `gR` referencias, `<leader>rn` renombrar, `<leader>ca` code action, `<leader>fm` formatear. Completar: `Ctrl-Space` en insertar. Dart lo arranca flutter-tools (Flutter SDK), no Mason.

trouble: `xx` lista de problemas, `xn`/`xp` siguiente/anterior error.

git: `gs` estado, `gn` siguiente hunk, `gsa` stage del hunk, `gc` commit, `gp` push. Diff grande: `gdo`. Conflictos de merge: `gco`/`gct`/`gcb`.

terminal / runner / tests: `tt` terminal, `jj` para salir del modo terminal. `r` ejecuta. `ten` test más cercano (si el runner del lenguaje existe).

DAP: `db` breakpoint, `dc` continuar, `du` UI. Sin adaptador instalado no hace magia.

Flutter: `fla` run, `flr` reload, `fls` restart, `flq` quit.

Copilot / AI: CopilotChat `cc`. Visual `<leader>as` envía la selección al agente que elijas con tu instrucción, y el agente empieza a trabajar. Siempre queda una copia en el registro `a`.

undotree: `<leader>u` en normal es el árbol de undo. En visual, `<leader>u` convierte a minúsculas; no activa el árbol.

12. IdeaVim

Plugins del IDE -> IdeaVim -> menú **Tools | Vim**. Copia `.ideavimrc` a `~/.ideavimrc`. Recarga con `:source ~/.ideavimrc`.

Los prefijos coinciden. No copies el `.vimrc` clásico dentro de IdeaVim (lleva vim-plug). Atajos de IA y CopilotChat no están en el IDE. Si un `<Action>` no existe: `:actionlist`.

13. Cuando se rompe

Qué ves	Qué mirar
Recuadros con `?` en vez de iconos	Fuente del terminal en JetBrainsMono Nerd Font Mono (README, Iconos)
Plugin en rojo	`:Lazy`
Completar vacío	`:Mason` y `:checkhealth vim.lsp`
Dart mudo	Flutter en PATH; no dupliques dartls
El atajo “no va”	Prefijo de 300 ms; `:Telescope keymaps`
`E37` al salir	hay cambios; `:wq` o `:q!`

Orden sugerido en clase

Cada sesión: presentación del módulo, práctica en la página y, desde el capítulo 8, la configuración real.

1. Mecanografía + `nvim --clean +Tutor` (capítulo 0, una sesión).

2. Modos, insertar, `:w` `:q` y ayuda (capítulo 1, lección 1).

3. Moverse sin flechas, números y `f`/`t` (capítulo 2, lección 2).

4. Gramática `d`/`c`/`y` + movimientos, deshacer (capítulo 3, lección 3).

5. Objetos `ciw` `ci"` `ci{` (capítulo 4, lección 4).

6. Buscar y `:s` (capítulo 5, lección 5), registros y una macro (capítulo 6, lección 6).

7. Buffers, ventanas y terminales (capítulo 7, lección 7).

8. Instalar esta config y usarla como IDE: Telescope, git, LSP (capítulos 8 y 9, lección 8).

9. IA con Neovim, plugins, IdeaVim y diagnóstico (capítulos 10 a 13).

Si alguien pide “el atajo de buscar archivo” el día 1, se puede enseñar `<leader>ff`, pero que sepa que es azúcar. El editor de verdad es el capítulo 3.

Grabación Udemy (secciones, vídeos, slides, checklist): UDEMY.md.

Evaluación práctica

Etapa	Evidencia para avanzar
Modos/edición	Crear un archivo, corregir una palabra, deshacer y guardar
Movimientos/objetos	Resolver `dw`, `ciw` y `ci"` y explicar la diferencia
Búsqueda	Sustituir con confirmación; distinguir rango de líneas y zona visual
Buffers/ventanas	Mostrar dos vistas del mismo buffer y volver por el historial
Registros/macros	Guardar en `a` y repetir un cambio sobre tres líneas
Config como IDE	Cliente LSP adjunto, definición y formato en un archivo de prueba
Git	Revisar un diff y preparar un commit local

Proyecto final: en un repo de práctica, cambia una función, ejecuta sus pruebas, revisa el diff y crea un commit local. No hace falta publicar ni hacer push. Guarda versión, comando y resultado.

Fuentes y mantenimiento

docs/SOURCES.md reúne fuentes oficiales. `python scripts/vim_reference.py --check` contrasta los casos del entrenador con Neovim; no prueba un emulador completo. Genera el PDF desde este archivo con `python scripts/build_course_pdf.py`. Evidencia y límites: docs/VALIDATION.md.